

МОСКОВСКИЙ ФИЗИКО-ТЕХНИЧЕСКИЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Физтех-школа аэрокосмических технологий

Отчёт о выполнении лабораторной работы «Контейнеры»

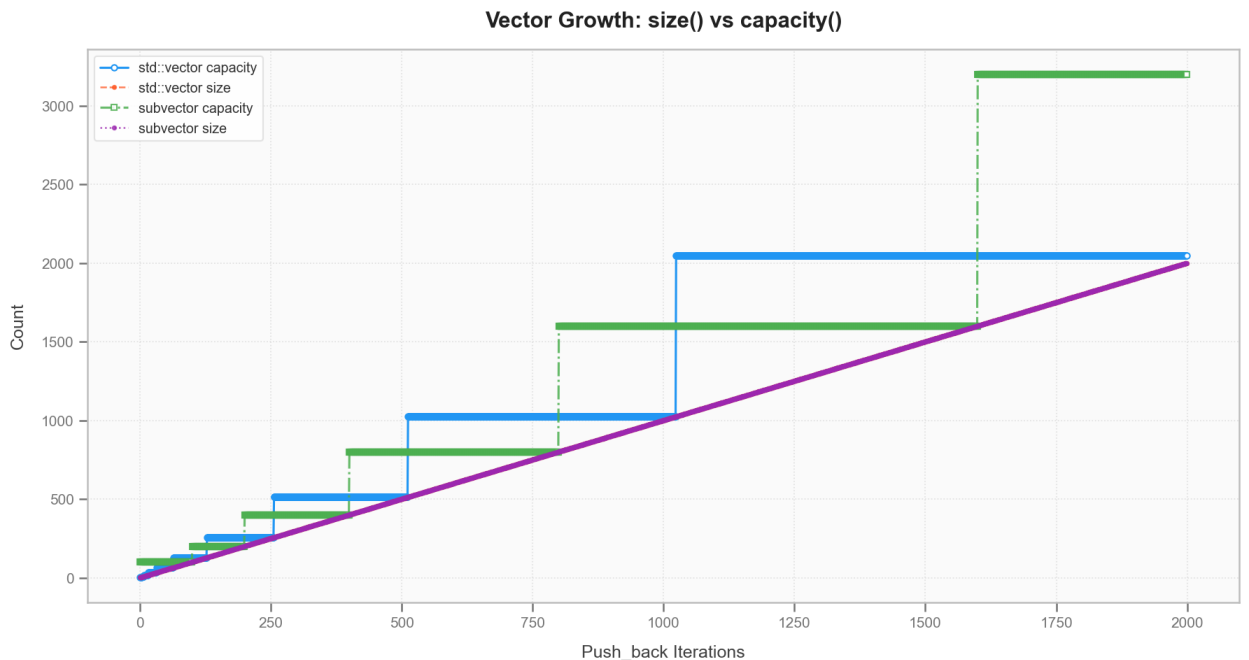
Автор:

Волков Илья Александрович

Б03-503

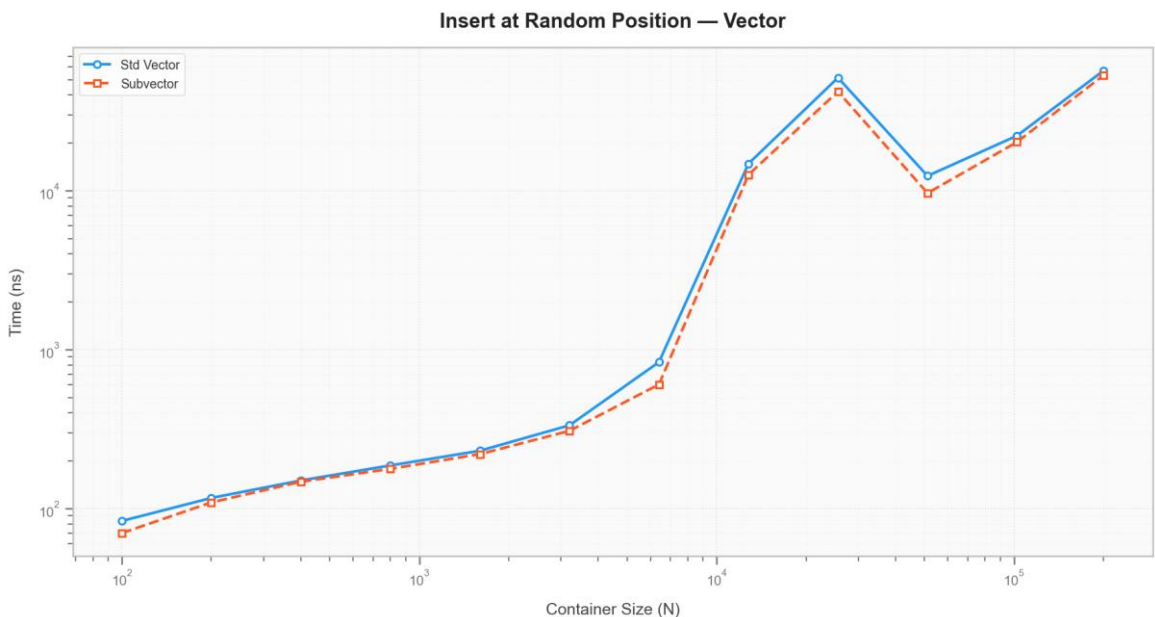
Долгопрудный 2025

0. Пример с лекции



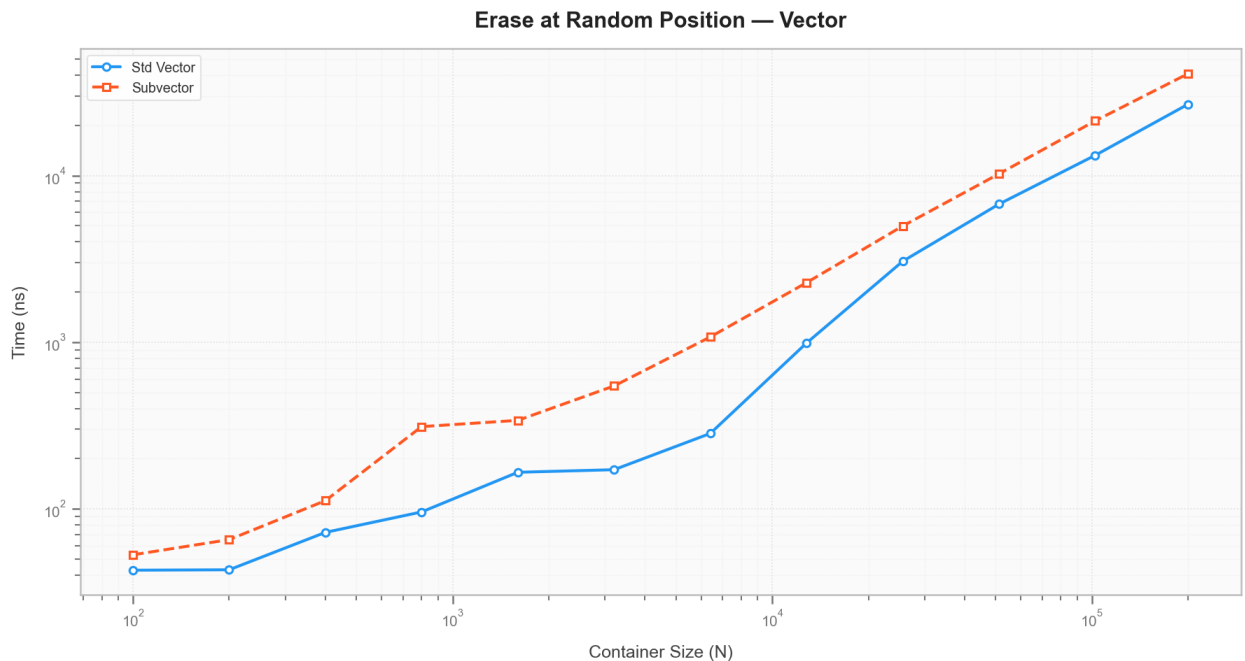
Оба контейнера (vector и subvector) расширяются геометрически: при заполнении текущего буфера выделяется новая область памяти удвоенного объёма, после чего данные копируются туда. Различия на графиках объясняются тем, что subvector инициализируется с начальным значением capacity = 100. В остальном поведение контейнеров полностью совпадает.

1. Среднее время вставки в произвольное место вектора



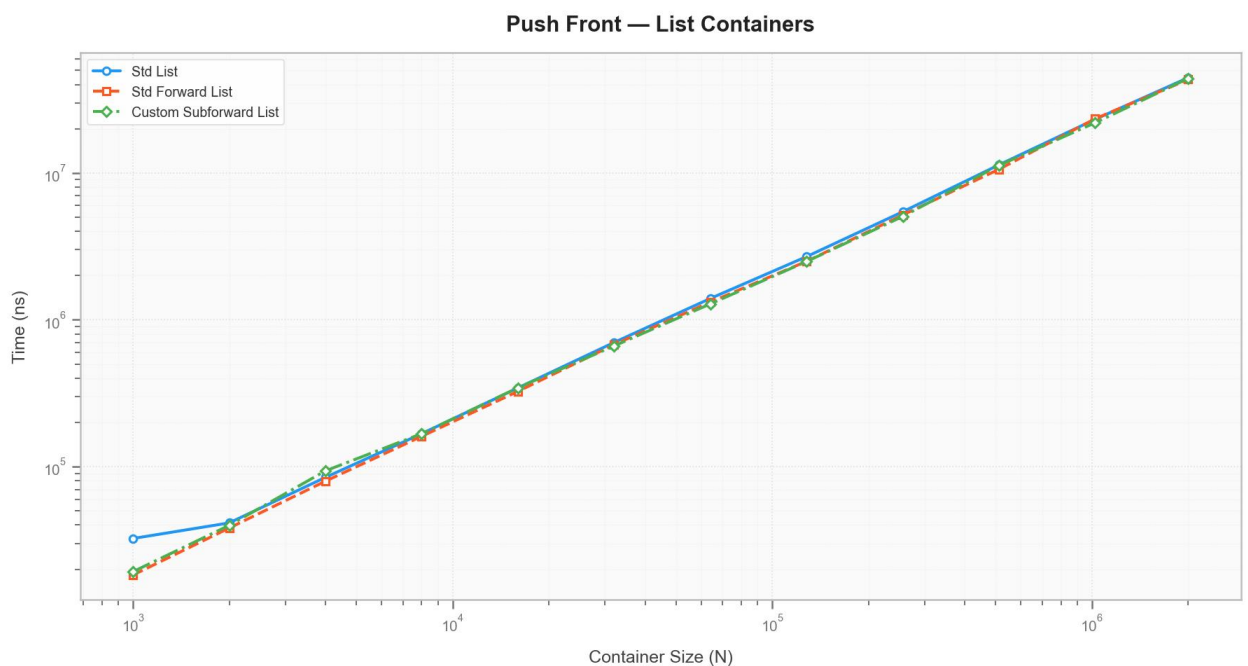
Графики для std::vector и subvector практически идентичны. Наблюдаемая линейная зависимость времени от размера подтверждает асимптотику $O(N)$: вставка требует сдвига всех последующих элементов. Локальные всплески на графике связаны со случайным характером выбираемой позиции вставки.

2. Среднее время удаления одного элемента из произвольного места вектора.



Удаление также демонстрирует сложность $O(N)$ из-за необходимости сдвига хвоста массива. При этом рукописная реализация показывает несколько худшие результаты по сравнению со стандартной — вероятно, из-за менее оптимальной реализации метода. Тем не менее, общая тенденция и форма графиков совпадают.

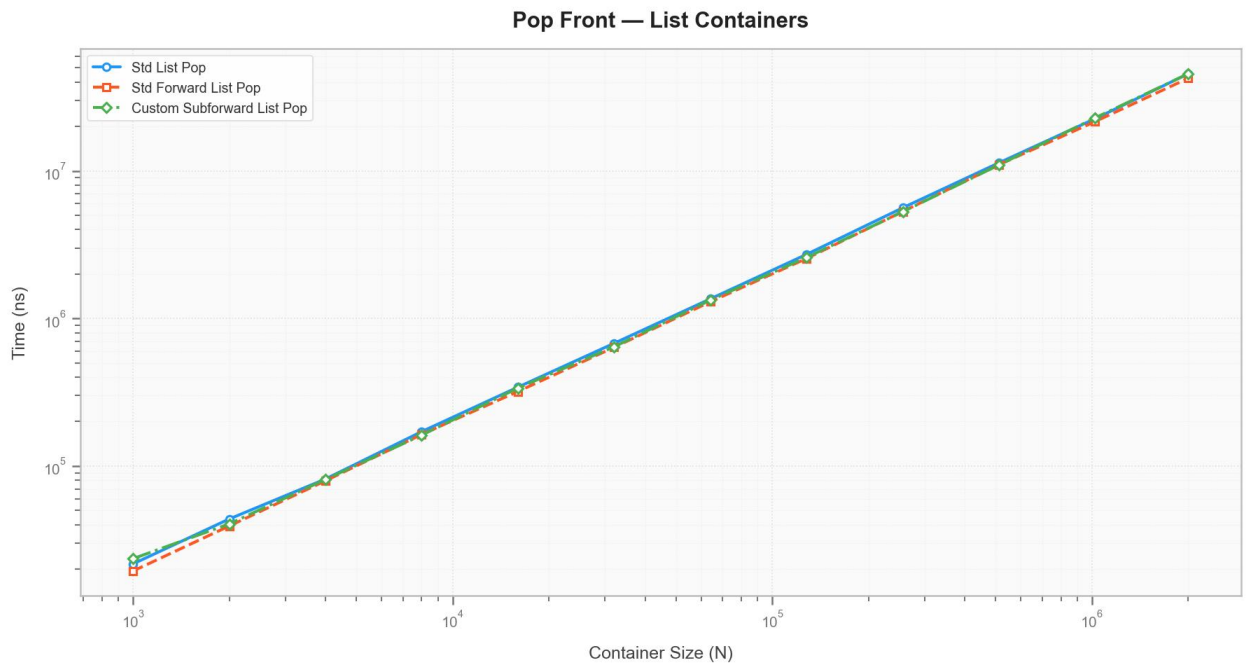
3. Среднее время добавления в начало односвязного списка.



Кривые для `std::list`, `std::forward_list` и `subforward_list` практически неразличимы. Линейный рост общего времени при выполнении N операций

указывает на константную сложность $O(1)$ для одной операции: достаточно создать новый узел и переназначить указатель на начало списка.

4. Среднее время удаления из начала односвязного списка.



Результаты аналогичны предыдущему пункту. Удаление первого элемента сводится к обновлению начального указателя и освобождению памяти старого узла, что также выполняется за $O(1)$.

5. Среднее время добавления элемента в бинарное дерево.

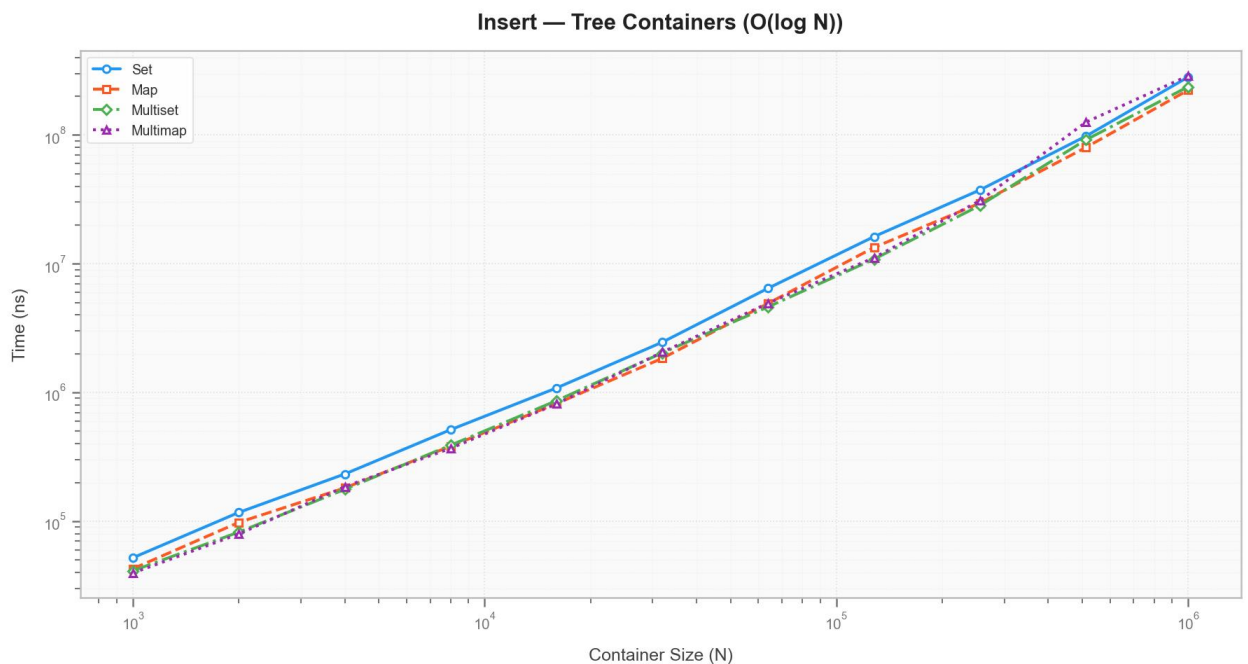
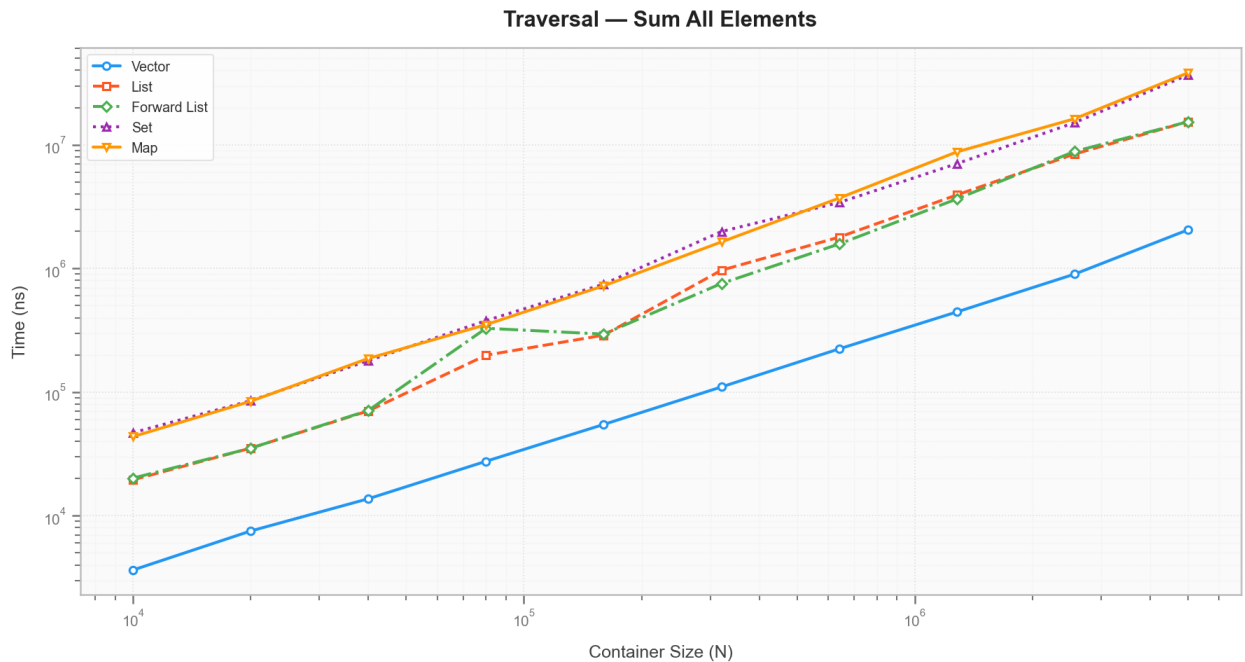


График имеет вид, близкий к прямой линии на логарифмической шкале. Поскольку измеряется время вставки N элементов, а каждая вставка требует $O(\log N)$, суммарная сложность составляет $O(N \cdot \log N)$.

- `std::set` (верхняя кривая) работает медленнее из-за дополнительной проверки уникальности ключа.
- `std::multiset` (нижняя кривая) быстрее, так как допускает дубликаты и избегает этой проверки.

6. Среднее время обхода всего контейнера.



На графиках с логарифмическими осями все зависимости идут параллельно, что подтверждает асимптотику $O(N)$ для всех типов контейнеров. Различия в абсолютном времени обусловлены не алгоритмической сложностью, а особенностями организации памяти:

- `Vector` — самый быстрый: данные расположены непрерывно, что обеспечивает высокую локальность и эффективное использование кэша.
- `List` / `Forward_list` — средняя скорость: узлы разбросаны в куче, разыменование указателей требует дополнительных обращений к памяти.
- `Set` / `Map` — самые медленные: каждый узел содержит ключ, значение и несколько указателей, что увеличивает объём данных и снижает эффективность обхода.

7. Среднее время доступа к произвольному элементу вектора

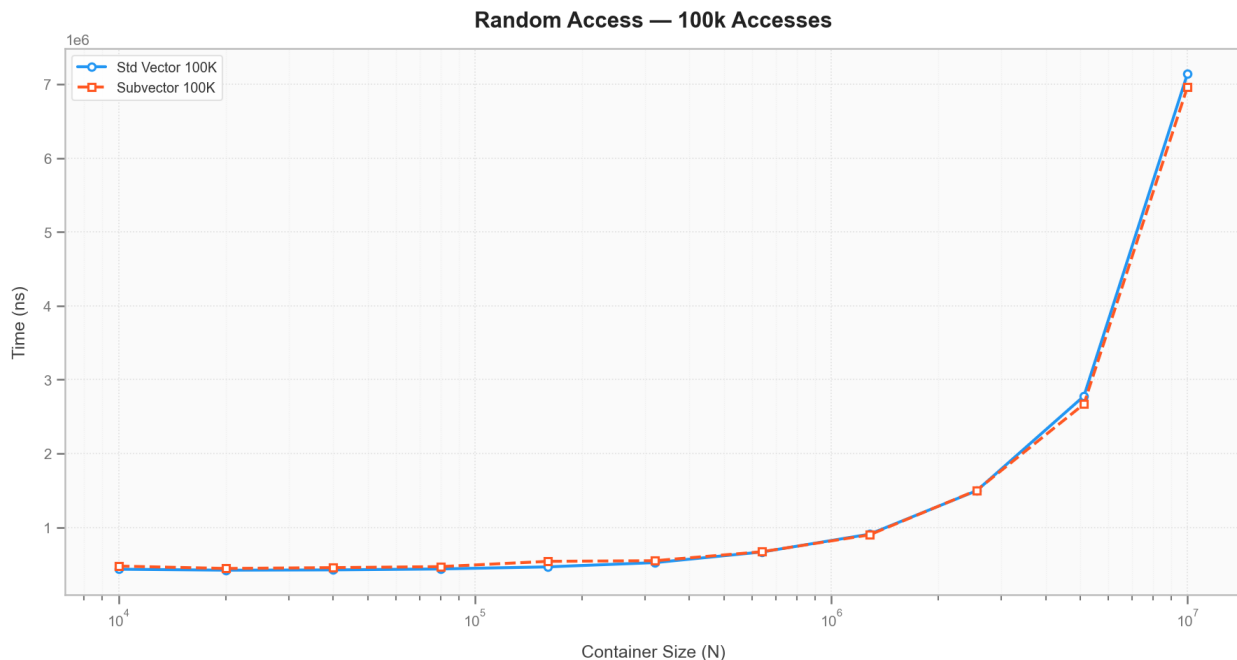


График отражает время выполнения 100 000 случайных обращений. До размера порядка 10^6 элементов рост времени незначителен — данные помещаются в кэш процессора, и доступ к ним происходит быстро.

Резкий скачок наблюдается в диапазоне 10^6 – 10^7 элементов: при размере `int` = 4 байта, 10^7 элементов занимают ~40 МБ, что превышает типичный объём кэша процессора (обычно ≤ 16 МБ). После этого порога процессор вынужден обращаться к оперативной памяти, что существенно увеличивает время доступа.